

C++

PENG

October 6, 2026

Contents

1	Introduction	1
1.1	IO	1
1.2	Comments	2
1.3	Flow of Control	2
1.4	Class	2
2	Types	3
2.1	Compound Types	3
2.2	Data structure	4
3	Vectors	4
3.1	String	4

1 Introduction

C++ is a high-level, statically typed programming language [1].

1.1 IO

Definition 1.1 (IO objects).

- `iostream` : *std lib*
- `std::cin` : *istream type*
- `std::cout` : *ostream type*

```
std::cout << "str";           // return object: std::cout
std::cout << "str" << "\n"; // (std::cout << "str") << "\n";
```

```
int v1 = 0, v2 = 0;
std::cin >> v1;           // return object: std::cin
std::cin >> v1 >> v2;     // (std::cin >> v1) >> v2;
```

Definition 1.2 (namespace).

`namespace::name` // `::` scope operator

```
// headers file should not include using declarations
using namespace::name;
```

1.2 Comments

Definition 1.3 (comment).

```
// single line, best way
/* paired, can not nest */

std::cout << /* */ /* */ /* */;
// remove everything between */ and closest /* before
// std::cout << " /* ";
```

1.3 Flow of Control

Definition 1.4 (while).

```
while(cond)
{
    statements;
}

while(std::cin >> val)
// std::cin >> val return left operand: std::cin
// istream become invalid:
// 1. reading a non-integer val: 1 2 3.1
// 2. end-of-file: Ctrl-c
```

Definition 1.5 (for).

```
for (init-statement; cond; expr)
{
    statements;
}

// init-statement:
// local var used only inside for, executed only once
// expr: executed after for body
```

Definition 1.6 (if).

```
if (cond)
{
    statements;
}
else if (cond)
{
    statements;
}
else
{
    statements;
}

if (std::cin >> val)
    std::cout << val << "\n";
// 1 2 3 RET => 1
```

1.4 Class

Definition 1.7 (class). *Define data structures:*

- *name*

- *defined*
- *operation*

```
// #include <std lib>
// #include "header.h"
```

Definition 1.8 (member function).

```
data.func()
```

2 Types

Definition 2.1 (arithmetic types).

```
char          // '\0': null, '\t': horizontal tab, '\n': newline
#include <string>
std::string // "A": ['A', '\0']
int          // unsigned: >= 0
bool         // true for 1 and false for 0
double
```

Definition 2.2 (object). *An object is a region of memory that contain data and has a type.*

Definition 2.3 (declaration and definition).

```
// declare: specify a base type and a list of var names
// can be declared many times
// main file:
#include "header.h"
extern int i;

// define: allocate storage and may provide initial val
// defined exactly only once
// header file:
int i; // 0
```

Definition 2.4 (scope). *Delimited by {}.*

```
// vars defined outside any function body are initialized to 0
int val; // 0
int main ()
{
    int val = 1; // block scope
    // ::val == 0 global scope
    ...
}

// var: lowercase
// class: first letter uppercase
```

2.1 Compound Types

Definition 2.5 (reference). *A reference defines an alias for an object.*

```
int i = 0;
int &ref = i;
// ref must be bound only to an int object, not a literal
// error: int &ref = 1;
```

Definition 2.6 (pointer). *A pointer is an object that holds the address of another object.*

```

int i = 0;
int *p;
p = &i;      // p holds the address of i
*p = 0;      // return the object i to which p points
p = nullptr; // or p = 0, null pointer, not bound to any object

// pointer to pointer to ...
int **pp;    // * indicate pointer level
pp = &p;

// ref to pointer
int *&ref = p; // & => ref is reference, * => ref refers to pointer type

```

2.2 Data structure

Definition 2.7 (structure). *A data structure is a way to group together related data elements and for using those data.*

```

// class is ordinarily defined outside functions, usually in className.h
struct className
{
    statements;
}; // must end with ;

className x, *y;

```

Definition 2.8 (preprocessor). *Headers should have guards.*

```

// a header may be included multiple times:
// className.h: #include <string>
// main: #include <string>
// to be safe to include a header multiple times:

// className.h
#ifndef CLASSNAME // true if CLASSNAME not defined
#define CLASSNAME // preprocessor var are all uppercase
struct className {};
#endif

```

3 Vectors

3.1 String

Definition 3.1 (string). *A string is a variable-length sequence of characters.*

```

#include <string>
using std::string;

```

References

- [1] S.B. Lippman, J. Lajoie, and B.E. Moo. *C++ Primer*. Pearson Education, 2012. ISBN: 9780133053036.